

skycell: a cost-based HEALPix index for sky positions in PostgreSQL

Design, brute-force validation, and a comparison with Q3C and pgSphere

J. Salgado¹  and M. Delli Veneri¹ 

SKA Observatory, Jodrell Bank, Lower Withington, Macclesfield SK11 9FT, United Kingdom
jesus.salgado@skao.int, michele.delliveneri@skao.int

Received ...; accepted ...

ABSTRACT

Context. Astronomical archives answer positional queries from relational databases, and in the PostgreSQL world two extensions dominate: Q3C, which keys each source by an integer drawn from a quad-tree on a cube and stores it in a B-tree, and pgSphere, which stores spherical geometry in a GiST tree. The first is compact and fast for points; the second is general and indexes regions as well.

Aims. We ask whether the two can be combined: a single index that keeps the integer key and the B-tree, yet answers cones, polygons, cross-matches and stored footprints, and that speaks the geometry language archives already use.

Methods. We present **skycell**, a PostgreSQL extension that keys positions by their order-29 HEALPix cell in the nested scheme and turns any query region into a small set of index ranges. How finely to cut is decided by a cost model – the price of an extra index range against the rows it removes – with the local source density read from the histogram **ANALYZE** already keeps for the key column, which is a density map precisely because HEALPix cells are equal area. Cells are classified against the region by their own corner geometry, the cell order follows in closed form, and a planner support function rewrites the predicate into B-tree range conditions plus an exact test before path generation. Correctness is established by brute force: coverings are checked against millions of catalogue points and against points sampled inside adversarial cones. Performance is measured on a 10-million-source synthetic catalogue with Gaia-like crowding, against Q3C 2.0.5 and pgSphere 1.5.2 on identical data, and in an A/B test inside a working TAP service.

Results. On the synthetic catalogue the median cone search is faster than both extensions at every radius tested, from 1'' to 3° (for example 0.024 ms against 0.035 ms for pgSphere and 1.330 ms for Q3C at 1'', and 0.573 ms against 1.010 ms and 3.023 ms at 3°), while touching about half of pgSphere's buffer pages at large radii and keeping an index of 214 MB against pgSphere's 683 MB. Convex polygons are four times faster than pgSphere's, a 200 000-probe cross-match at 1'' takes 0.47 s against 1.95 s for Q3C and 2.85 s for pgSphere, and the planner's row estimates are 1.2 times off instead of 1.9–2.8. A control measurement that interleaves the two methods query by query, with skycell measured first, reduces the cone-search advantage to a factor 0.61–1.04. On a 500 096-row ObsCore table inside a TAP server the same build is 1.2–1.45 times *slower* per query than pgSphere, and all 18 end-to-end comparison cells tie.

Conclusions. A cost-based covering over an equal-area integer key is competitive with both established designs on large catalogues and keeps the generality of a geometry type, at the price of planning work that only pays off once the scan dominates the query. The crossover in these measurements is a few million rows.

Key words. Astronomical databases: miscellaneous – Catalogs – Virtual observatory tools – Methods: data analysis

1 Introduction

Almost every question asked of an astronomical archive is positional: what is within this cone, which observations cover this field, which of my sources match that catalogue. In the Virtual Observatory these questions arrive as ADQL (Ortiz et al. 2008) through the Table Access Protocol (Dowler et al. 2019), and the service has to turn them into something a relational database can answer with an index.

Two PostgreSQL extensions carry most of this traffic. Q3C (Koposov & Bartunov 2006) projects the sphere onto a cube, subdivides each face with a quad-tree, and numbers the cells along a space-filling curve; each source is one 64-bit integer in a B-tree, and the table can be sorted along the same curve so that neighbours on the sky are neighbours

on disk. pgSphere (Chilingarian et al. 2004) takes the other route: it adds spherical types – points, circles, polygons, boxes – and indexes them with GiST over bounding volumes, so that regions can be stored and queried as data, not only as query arguments.

Neither idea is new to astronomy. The Hierarchical Triangular Mesh (Kunszt et al. 2001) had already keyed positions by a subdivided polyhedron, and the zones algorithm (Gray et al. 2006) showed how far a purely relational arrangement – declination strips and a B-tree – can be pushed for cross-matching. What has changed since is the size of the catalogues and the fact that most queries now arrive through a generated layer (e.g. Demleitner et al. 2014) rather than being written by hand, so the index has to serve a translator's output rather than a tuned query.

The two designs trade off against each other. A

linear key in a B-tree is compact, cheap to probe and orders the heap, but a region has to be turned into a set of key ranges before the index can be used, and a set of ranges is an approximation whose quality depends on how finely the region is cut. A tree of bounding volumes has no such approximation step and handles extended objects naturally, but its keys are larger, its nodes overlap, and it imposes no useful order on the table.

This paper presents `skycell`, which keeps the integer key and asks a cost question at the point where the classical approach fixes a constant: how finely should a region be cut? The answer depends on two quantities the database already knows – how many sources sit in that part of the sky, and how much an extra index range costs – and it can be computed rather than configured. Section 2 develops the method, Sect. 3 describes what it takes to make a query planner use it, Sect. 4 the brute-force validation, Sects. 5 and 6 the measurements against Q3C and pgSphere, and Sect. 7 an A/B test inside a working TAP service, which is where the method’s limits show.

2 Method

2.1 An equal-area integer key

A position is keyed by the HEALPix cell containing it at order 29 in the nested scheme (Górski et al. 2005), a 64-bit integer addressing cells of about 0.4 mas. Three properties of that choice matter, and only the first is shared with the cube quad-tree of Q3C.

Cells are *nested*: the descendants of a cell at any order form one contiguous interval of order-29 ids, so a region covered by cells becomes a union of integer ranges, and a B-tree can answer it.

Cells are *equal area*. Every cell subtends the same solid angle, so the count of keys in an interval divided by the interval’s length is proportional to a source density per unit area. The histogram that `ANALYZE` keeps for the key column is therefore a map of how crowded each part of the sky is, at the resolution of its buckets, without any extra statistics machinery.

Cells are *standard*. The same numbering underlies Gaia’s `source_id` (Gaia Collaboration 2016) and the IVOA’s Multi-Order Coverage maps (Fernique et al. 2014), so keys, coverings and stored footprints are interoperable rather than private to one extension.

2.2 Coverings decided by cost

A query region is covered by cells whose union contains it; the covering becomes range conditions on the key, and rows inside the covering but outside the region are removed by an exact test. Cutting finer means fewer such false positives and more ranges.

With cells of size s covering a region of characteristic radius r and a local source density ρ (sources per unit solid angle),

$$C(s) = n_{\text{ranges}}(s) c_{\text{range}} + \rho (A_{\text{cover}}(s) - A_{\text{region}}), \quad (1)$$

where c_{range} is the price of one extra index range expressed in rows. A covering follows the region’s boundary, so for $s \ll r$ the number of ranges grows as r/s – the curve enters and leaves the region once per boundary crossing, and the number of such clusters is the classical measure of a space-filling curve’s quality (Moon et al. 2001) – while the wasted area grows as the perimeter times s ; minimising Eq. (1) then gives

$$s_{\star} = \sqrt{\alpha c_{\text{range}} / \rho}, \quad (2)$$

which does not depend on the size of the region: the density of the sky and the price of a range decide how finely it is worth cutting. For $s \gtrsim r$ the covering is a handful of cells and the balance differs, so rather than solve the branches the implementation evaluates Eq. (1) at all 30 orders – a few floating-point operations each – and takes the cheapest, which also lets a cap on the number of ranges and a guard on cell size simply remove candidates. The density ρ is read from the histogram around the region’s own centre, so a crowded field is cut finer than empty sky with no refinement loop at all.

Figure 1 shows the two extremes on the same cone radius: at high Galactic latitude the covering is a single range of large cells covering 5.6 times the cone’s area, because there is almost nothing to exclude; in the bulge the same cone is covered by 23 cells in 8 ranges covering 1.3 times its area, because there the rows removed are worth the extra ranges.

Two guards keep the model honest. A histogram is a coarse synopsis, and a dense cluster split by the space-filling curve into pieces smaller than one bucket can be underestimated; a partly covered cell is therefore never allowed to exceed a multiple (64 by default) of the region’s own area, which bounds the damage to the density-proportional behaviour a fixed-depth covering would have had. Gaps between ranges are merged when the rows they contain cost less than the range they save.

2.3 Cells tested by their own geometry

Whether a cell is inside, outside or on the boundary of the region has to be decided conservatively – a cell wrongly called outside loses rows – but not crudely, because every unnecessary cell is false positives at query time. Approximating each cell by a cap of the largest pixel radius found anywhere on the sphere, the simplest safe choice, is loose by a large factor for most cells.

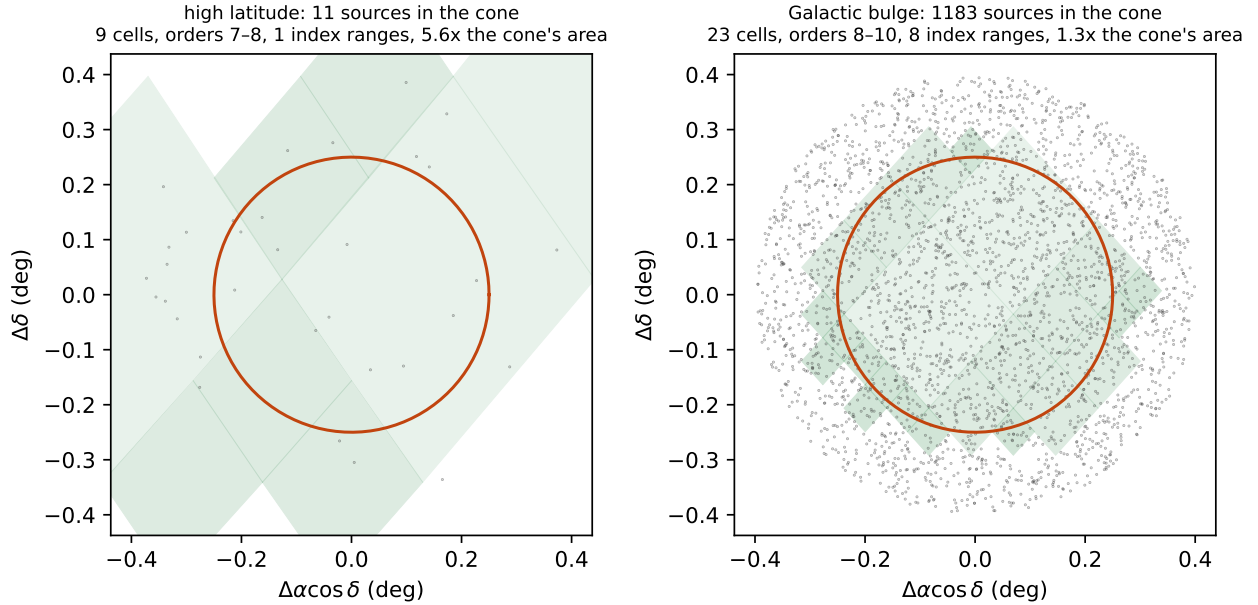


Figure 1: The same 0.25° cone covered in two places on the same catalogue. *Left*: at high Galactic latitude the cost model buys one index range of large cells, covering 5.6 times the cone’s area, because the 11 sources in the cone leave almost nothing to exclude. *Right*: in the Galactic bulge the same cone holds 1183 sources, and the model cuts two orders deeper – 23 cells in 8 ranges, 1.3 times the cone’s area – because there the false positives cost more than the ranges save. Grey dots are catalogue sources, the heavy circle the query cone, and the shaded quadrilaterals the covering cells, drawn from their own corners.

`skycell` instead uses the cell’s own four corners, computed from the continuous HEALPix face coordinates, and the great-circle chords between them. A cell lies inside a cone when its farthest corner does; it lies outside when the distance from the cone’s centre to every chord exceeds the radius. The chords are not the cell’s edges, which bow outwards, so each edge’s departure from its chord is measured for that cell from the edge midpoint. This matters at the poles, where an edge spans a large range of azimuth and departs from its chord by a fraction of the cell’s size rather than by its square: a global constant fitted away from the poles produces false negatives there, which is exactly what the validation of Sect. 4 caught during development.

2.4 Regions as stored objects

A footprint is covered by a small number of cells written in the NUNIQ encoding of the MOC standard and stored one per row in an ordinary B-tree. The footprints containing a given position are those whose cell is one of that position’s ancestors, a short list of equality lookups; the exact geometry is then applied to the candidates. Since a MOC’s cells are disjoint by construction, no duplicate elimination is needed. Refinement of these coverings stops when cells become negligible next to the region, which keeps the ancestor list short: without that stop, boundary cells whose siblings all fall outside keep splitting at no cost in cell count, and the lookup pays for orders that

carry almost no area.

2.5 The query surface

The extension exposes ADQL’s own vocabulary: types `skypos` and `skyregion` and functions `point`, `circle`, `box`, `polygon`, `contains`, `intersects`, `distance`, `area`, `coord1`, `coord2` and `centroid`, so that a translator emits the standard’s spelling rather than a dialect. The operator form `pos <@ circle(...)` is the one a planner can answer from an index; the function form returns the same rows.

Beside the geometry, archives need the astrometry that decides *where* a source is at the epoch being asked about. `skycell` implements the IVOA UDF registry’s names – `ivo_epoch_prop`, `ivo_epoch_prop_pos`, `ivo_apply_pm`, `ivo_healpix_index`, `ivo_healpix_center` – with the rigorous epoch propagation of the Hipparcos catalogue (ESA 1997), in which the star follows a straight line in space so that parallax, proper motion and radial velocity all evolve together. Propagating a six-parameter solution forward and back returns the starting position to better than a microarcsecond (Sect. 4). Frame conversions between ICRS, Galactic and ecliptic coordinates are provided for the same reason.

An index built on catalogue positions cannot know where a star has moved to, so a proper-motion-aware cone search is written as an indexable cone widened by the largest motion of interest, intersected with an

exact test on the propagated position. Table 1 lists the surface.

3 Implementation

The extension is written in C against the PostgreSQL 15+ server API and consists of a geometry layer with no database dependencies – HEALPix arithmetic, region predicates and the covering – and a layer that connects it to the planner.

The connection is a *planner support function* (`SupportRequestSimplify`), the mechanism PostGIS uses for `ST_DWithin`. Before path generation, a predicate such as `point('ICRS', ra, dec) <@ circle('ICRS',  $\alpha$ ,  $\delta$ ,  $r$ )` is replaced by a disjunction of range conditions on the key expression, conjoined with the exact test:

```
(cell BETWEEN lo1 AND hi1
OR cell BETWEEN lo2 AND hi2 OR ...)
AND skycell_in_region(point(ra, dec),
                      circle(...), 0.29)
```

The key expression is not written by the user: the support function builds the candidate expression from the predicate’s position argument and keeps it only if the relation actually carries an index on it, so the same query text works whether positions are stored as two columns or as a `skypos` column. Because the index conditions are then ordinary range predicates, the planner estimates their selectivity from the same histogram, and the last argument of the exact test carries the fraction of the covering that the region occupies, so that the product of the two estimates is close to the truth. This is why the row estimates in Sect. 6 are accurate, and accurate estimates are what keep a spatial predicate from being joined in the wrong order in a larger query.

Two caches matter for planning cost. The density model and the identity of the matching index are cached per backend and dropped whenever statistics, the relation or the relation cache change; coverings are memoised by region and statistics source, since a service plans the same cone many times. Measured on a real corpus, the first cache is worth 45–70 μ s per plan and the second up to 50 μ s for the larger regions.

For joins, where the region is not a constant, the rewrite emits a fixed number of range slots computed per outer row – the structure `q3c_join` uses – while a set-returning function `skycell_cone_ranges()` offers the alternative of a variable number of ranges through a `LATERAL` join. The two differ in plan shape rather than geometry, and Sect. 6 shows the difference is large.

4 Validation

A covering that misses a cell silently loses rows, so correctness is established by brute force rather than by inspection, at three levels.

The HEALPix layer is checked against known cell centres, round trips between positions and cells at all 30 orders, the nesting property, equal area, and the bound the covering relies on – that no point of a cell lies farther from its centre than its farthest corner – over 1.6 million sampled points including the poles, the boundary between the polar and equatorial zones, and the base-pixel edges.

The covering is checked against a 2-million-point clustered catalogue for cones from 1" to 120° and for convex polygons, and against 30 000 adversarial cones – at the poles, across the zone boundary, at $\alpha = 0$ and on face edges, with radii from 0.02 mas to 10° – with 200 points sampled inside each, half of them within a part per billion of the rim. Both assert zero false negatives. This is the test that caught the polar edge-bulge error of Sect. 2.

At the SQL level, two regression suites require that every indexed answer equals the answer from a sequential scan with the exact predicate: 240 cones in two configurations, 60 polygons, both join forms and the MOC footprint lookup in the first; the ADQL surface, the operator and function spellings over 120 random regions, the presence of index plans (and their absence when the region is not constant), epoch-propagation round trips and frame conversions in the second.

5 Benchmark

5.1 Corpus

The catalogue is 10 million synthetic sources with a density contrast resembling a real optical survey: 35% uniform on the sphere, 45% in a Galactic disk (longitude uniform, latitude exponential with a 3° scale), 12% in a bulge (Gaussian, 7°), and 8% in 200 clusters with dispersions from 0.02° to 0.5°. Positions are drawn deterministically from a seed. Peak densities exceed the mean by more than three orders of magnitude, which is what makes the density model’s job non-trivial.

5.2 Targets

Q3C 2.0.5, pgSphere 1.5.2 and `skycell` each hold the same logical rows in their own natural layout: Q3C an expression B-tree on `q3c_ang2ipix` with the table sorted along the same curve; pgSphere an `spoint` column with a GiST index; `skycell` a `bigint` cell column with a B-tree and a 1000-bucket histogram. All three tables are physically ordered along a space-filling curve, so heap locality is held equal. Queries are the documented interface of each

Table 1: The query surface. ADQL spellings are used unchanged; the `skycell`-prefixed aliases exist for schemas where a name collides with one in `pg_catalog`.

Signature	Meaning
<i>Geometry (ADQL)</i>	
<code>point(frame, ra, dec)</code>	position → <code>skypos</code>
<code>circle(frame, ra, dec, r)</code>	cone → <code>skyregion</code>
<code>box(frame, ra, dec, w, h)</code>	box → <code>skyregion</code>
<code>polygon(frame, ra, dec, ...)</code>	convex polygon
<code>contains(a, b), intersects(a, b)</code>	1 or 0
<code>distance(a, b), area(r)</code>	degrees, square degrees
<code>coord1/coord2/centroid</code>	components, centre
<code>pos <@ region</code>	indexable containment
<i>Astrometry (IVOA UDF registry)</i>	
<code>ivo_epoch_prop(...)</code>	six parameters propagated
<code>ivo_epoch_prop_pos(...)</code>	position only
<code>ivo_apply_pm(...)</code>	proper motion only
<code>ivo_healpix_index/center</code>	cell of position, and back
<code>icrs2gal, gal2icrs</code>	frame conversion
<code>icrs2ecl, ecl2icrs</code>	frame conversion
<i>Index and covering</i>	
<code>skycell_ang2cell(ra, dec)</code>	the order-29 key
<code>skycell_cone/poly(cell, ...)</code>	indexable predicates
<code>skycell_cone_ranges(...)</code>	ranges as rows (<code>LATERAL</code>)
<code>skycell_cone_moc(...)</code>	covering as <code>NUNIQ</code> cells
<code>skycell_ancestors(cell, ...)</code>	a position's containing cells

extension: `q3c_radial_query`, `spoint <@ scircle` and `skycell_cone`.

5.3 Protocol

Each measurement is a fixed set of query centres, half drawn from catalogue positions (hence density-weighted) and half uniform on the sky, run once to warm and once to measure, with a third pass under `EXPLAIN (ANALYZE, BUFFERS)` to record buffers, planning time and the planner's row estimate against the actual count. Cone radii span $1''$ to 3° with 16–400 queries each. Cross-matches join a 200 000-row probe table – half catalogue sources displaced by $\sim 0.3''$, half uniform – to the catalogue. Footprint queries ask which of 20 000 stored circles contain each of the 200 000 probes. All row counts are compared between methods.

Everything runs on one machine: PostgreSQL 18.6 in a Docker Desktop virtual machine on an 11-core Apple Silicon laptop, 8 GB of memory, `shared_buffers` 2 GB, JIT off and parallel query disabled so that each measurement is one core doing one query. Absolute numbers are therefore properties of this machine; the comparisons are between methods measured on it within minutes of each other.

Two biases are worth naming. The corpus is synthetic and its density structure was chosen by the author of the method under test. And the three methods are measured in separate phases, so a phase measured later benefits from a warmer cache; the control in Sect. 6.5 quantifies that.

6 Results

All three methods return identical row counts on every query of the grid. In the cross-matches `pgSphere` returns one or two more pairs out of $\sim 100\,000$, consistent with its 10^{-9} rad comparison tolerance counting sources exactly on the rim.

6.1 Cone searches

Table 2 and Fig. 2a give the medians. `skycell` is fastest at every radius, by a factor 1.3–1.5 against `pgSphere` below $6'$ and 2.3 at 1° . Q3C is an order of magnitude slower on small cones for a structural reason: version 2.0.5 expands *every* `q3c_radial_query` into 100 key ranges regardless of radius, which costs about 300 buffer pages and 1.2 ms of planning before any row is read. Its cross-match path, which uses four ranges per probe, does not pay this.

The buffer counts (Fig. 2b) show where the difference comes from at large radii: at 1° `skycell` touches 55 pages against `pgSphere`'s 154, and at 3° 343 against 617, because a covering sized by the cost model wastes less area than a tree of bounding boxes whose leaves were built without reference to the query.

6.2 Polygons, cross-matches and footprints

Convex polygons of 0.05° – 2° take a median 0.142 ms against `pgSphere`'s 0.631 ms and Q3C's 1.526 ms; `pgSphere`'s mean (3.9 ms) and 95th percentile (20 ms)

Table 2: Cone searches on the 10-million-source catalogue: median query time (planning and execution), buffer pages touched, and the error of the planner’s row estimate (median of $\max(\text{est}/\text{act}, \text{act}/\text{est})$). All three methods return identical row counts on every query.

radius	$\langle \text{rows} \rangle$	median time (ms)			buffers			estimate error		
		Q3C	pgSphere	skycell	Q3C	pgSphere	skycell	Q3C	pgSphere	skycell
1''	0.5	1.330	0.035	0.024	301	5.8	3.9	1.00	1.00	1.00
10''	0.7	1.374	0.030	0.023	301	5.6	4.3	1.00	1.00	1.00
1'	3.4	1.366	0.032	0.024	301	6.4	5.3	1.00	1.00	1.00
6'	80.1	1.353	0.038	0.033	303	13.6	9.3	2.00	4.00	1.46
30'	516	1.373	0.095	0.075	313	61.4	30.1	1.92	2.92	1.16
1°	1 476	1.427	0.315	0.139	329	154	55.5	1.89	2.78	1.22
3°	20 652	3.023	1.010	0.573	649	617	343	2.32	2.93	1.10

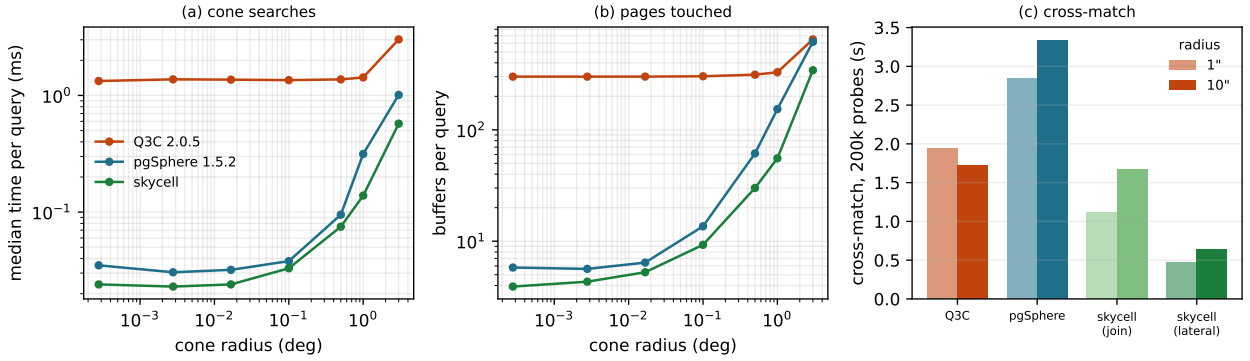


Figure 2: Cone searches and cross-matching on the 10-million-source catalogue. (a) median query time against cone radius, planning included; (b) buffer pages touched per query; (c) wall time of a 200 000-probe cross-match at 1'' and 10'', with the two plan shapes **skycell** offers for the same geometry. Note the logarithmic axes in (a) and (b).

Table 3: Storage and build cost for the same 10 million sources. Table sizes differ because each method stores its own position representation alongside the catalogue columns. **ANALYZE** is slower for **skycell** because the histogram is computed over an expression.

method	table (MB)	index (MB)	build (s)	ANALYZE (s)
Q3C	574	214	1.8	0.16
pgSphere	730	683	43.9	0.85
skycell	652	214	2.3	1.40

are far above its median, which the per-query records trace to polygons whose bounding volume is a poor fit.

Cross-matching 200 000 probes at 1'' takes 0.47 s with the **LATERAL** form, against 1.12 s for **skycell**'s own join form, 1.95 s for **q3c_join** and 2.85 s for **pgSphere** (Fig. 2c). The gap between the two **skycell** forms is plan shape, not geometry: the join form and **q3c_join** both build a bitmap from several bitmap index scans for every probe, while the **LATERAL** form issues plain range scans, as many as that probe's covering needs. With a covering index the same query becomes an index-only scan and 0.59 s at 10''.

For stored footprints, the MOC-in-a-B-tree arrangement answers 140 996 point-in-region pairs in 0.67 s against **pgSphere**'s 1.10 s, at the cost of an index that is larger (10 MB against 1.2 MB) and slower to build (1.1 s against 0.04 s), since a region becomes several rows rather than one.

6.3 Index size, build time and estimates

The B-tree of 64-bit keys is 214 MB and builds in 2.3 s; **pgSphere**'s GiST is 683 MB and builds in 44 s (Table 3). For an archive this is an operational number: it is what an ingest, a restore or a schema migration has to pay.

The planner's row estimate for a 1° cone is off by a median factor 1.22 for **skycell**, against 1.89 for **Q3C** and 2.78 for **pgSphere**, because the rewritten predicate is a range condition on an ordinary expression and the planner estimates it from the ordinary histogram.

6.4 Ablations

Switching off the density model, removing the area guard, or moving c_{range} between 3 and 30 changes small-cone times by a few percent on this catalogue: with cells much larger than the cone, the covering is one or two cells whatever the model says. The density model earns its place on medium cones in crowded fields, which is where Fig. 1 shows it cutting finer, and on the false positives that would otherwise be read.

6.5 An interleaved control

Because the grid measures methods in separate phases, we repeated the cone searches alternating the two methods query by query, warm, with **skycell** going first – the order that disadvantages it. The ratios of **skycell** to **pgSphere** are then 0.85, 1.04, 0.92, 0.72 and 0.61 at 1'', 1', 30', 1° and 3°: the advantage survives at every radius but one, where the two tie, with smaller margins than the phase-ordered grid. The grid should be read as the optimistic end of the range and this control as the conservative one.

7 Inside a TAP service

A benchmark of index operations is not a benchmark of an archive. We therefore ran the same build inside **egernia** (Delli Veneri & Salgado 2026), a TAP 1.1 server, against its published 500 096-row **ObsCore** corpus, in an A/B where one database held both indexes and two otherwise identical service processes differed only in how they translate **CONTAINS**: one to **pgSphere**, one to **skycell**. The service's own harness interleaved the two targets rung by rung.

Both targets pass the conformance gate (**taplint**, zero errors) and the cross-server agreement gate on all eleven portable query classes, and the 203 cone queries of the corpus return identical rows through both translations.

The performance result is a tie: all 18 comparison cells, across the mixed workload and five query classes at 1, 8 and 32 clients, fall inside the protocol's tie rule, with throughput ratios between 0.96 and 1.06. At the database level **skycell** is 1.2–1.45 times *slower* per query than **pgSphere** on that corpus.

Both facts have the same cause. A cone query over half a million rows returns tens to hundreds of rows and costs a fraction of a millisecond either way, against roughly 10 ms of CPU per request spent in the service's own Python; and at that relation size there is no scan left for a better covering to shorten, while the covering still has to be planned. The measurements put the crossover at a few million rows.

8 Discussion

The results support a narrow claim: on large catalogues, a covering sized by cost over an equal-area integer key is faster than both established extensions for the operations measured, with a smaller index and better row estimates, and it does not give up the ability to index regions. They do not support the claim that it is the better choice everywhere. On a small, already well-clustered relation it is slower, and inside a service whose cost is dominated by protocol handling it changes nothing at all.

What remains expensive is planning. A covering is computed per plan, and even cached it costs 0.01–0.12 ms, against 0.006 ms for a GiST predicate that needs no such step. The architectural way out is to stop planning ranges: an SP-GiST operator class over the cell hierarchy would present a single index qualification to the planner and refine during the index descent, where the work is proportional to the rows actually present rather than to the region. That would also remove the per-arm cost the planner pays for a disjunction of ranges, which our measurements put at 15–20 μ s each.

The density synopsis is the other limit. An equi-depth histogram over a space-filling curve is a poor two-dimensional density estimator: a cluster split by the curve into pieces smaller than one bucket is invisible to it, which is why the area guard exists. A custom `typanalyze` storing a multi-order density map – a MOC with counts – would estimate what the covering actually needs, and would also improve selectivity estimates for crowded fields.

Finally, nothing here has run on real data at Gaia density, with cold caches, or on more than one machine, and the corpus was designed by the same author as the method.

9 Conclusions

`skycell` keeps Q3C’s compact integer key and B-tree while answering the region queries `pgSphere` was built for, by treating the depth of a covering as a cost question whose answer follows from the local source density and the price of an index range. On a 10-million-source catalogue it is the fastest of the three for cone searches at every radius tested, four times faster than `pgSphere` for convex polygons, four times faster than Q3C for cross-matching, faster for point-in-footprint queries, and it keeps an index a third the size of `pgSphere`’s with row estimates twice as accurate. An interleaved control puts the cone-search advantage at 0.61–1.04 times `pgSphere`’s time rather than the larger factors of the phase-ordered grid.

The same build is slower on a 500 000-row table and makes no difference to a TAP server’s published throughput, because there the database is a small part of a request. An index is worth choosing carefully when the scan dominates; the measurements here put that crossover at a few million rows, which is where survey archives already are.

Data availability

The extension, its test suites, the benchmark harness and the raw results of every measurement quoted here are available at <https://github.com/jesusjuansalgado/skycell> under the MIT licence. The catalogue is generated from a seed by

`bench/01_data.sql`, so the corpus is reproducible without transferring data; `bench/run.sh` reproduces Tables 2 and 3 and Figs. 1 and 2 end to end. The TAP A/B of Sect. 7 is recorded, protocol and results, in the same repository.

Acknowledgements. The implementation, the benchmark harness and the analysis presented here were developed with the assistance of Anthropic’s Claude Code. All measurements were produced by the software cited in the data availability section and can be reproduced from it.

References

- Chilingarian, I., Bartunov, O., Richter, J., & Sigaev, T. 2004, in ASP Conf. Ser. 314, *Astronomical Data Analysis Software and Systems XIII*, ed. F. Ochsenbein, M. G. Allen, & D. Egret (San Francisco: ASP), 225
- Delli Veneri, M., & Salgado, J. 2026, *A&A*, submitted
- Demleitner, M., Neves, M. C., Rothmaier, F., & Wambsganss, J. 2014, *Astron. Comput.*, 7–8, 27
- Dowler, P., Rixon, G., Tody, D., & Demleitner, M. 2019, *IVOA Recommendation: Table Access Protocol Version 1.1*
- ESA 1997, *The Hipparcos and Tycho Catalogues*, ESA SP-1200
- Fernique, P., Boch, T., Donaldson, T., et al. 2014, *IVOA Recommendation: MOC – HEALPix Multi-Order Coverage map Version 1.0*
- Gaia Collaboration, Prusti, T., de Bruijne, J. H. J., et al. 2016, *A&A*, 595, A1
- Górski, K. M., Hivon, E., Banday, A. J., et al. 2005, *ApJ*, 622, 759
- Gray, J., Nieto-Santisteban, M. A., & Szalay, A. S. 2006, *The Zones Algorithm for Finding Points-Near-a-Point or Cross-Matching Spatial Datasets*, Microsoft Research Technical Report MSR-TR-2006-52
- Koposov, S., & Bartunov, O. 2006, in ASP Conf. Ser. 351, *Astronomical Data Analysis Software and Systems XV*, ed. C. Gabriel, C. Arviset, D. Ponz, & S. Enrique (San Francisco: ASP), 735
- Kunszt, P. Z., Szalay, A. S., & Thakar, A. R. 2001, in *Mining the Sky*, ed. A. J. Banday, S. Zaroubi, & M. Bartelmann (Berlin: Springer), 631
- Moon, B., Jagadish, H. V., Faloutsos, C., & Saltz, J. H. 2001, *IEEE Trans. Knowl. Data Eng.*, 13, 124
- Ortiz, I., Lusted, J., Dowler, P., et al. 2008, *IVOA Recommendation: Astronomical Data Query Language Version 2.0*